

PRÁCTICA 1

MANUAL DE USO DE LAS DEVTOOLS DEL NAVEGADOR

Nombre: Cristian Rodríguez Hidalgo

Curso: 2º DAW

Módulo: Desarrollo web en entorno cliente

Fecha: 01/10/2026

ÍNDICE

1. Introducción	3
2. Banco de pruebas: código HTML, CSS y JavaScript	3
3. Elements / Inspector	5
3.1. Inspección y edición del DOM	5
3.2. Edición del CSS en vivo	5
3.3. Selector de elementos y estilos computados	6
3.4. Edición de atributos y clases	6
4. Console	7
4.1. console.log()	7
4.2. console.error()	7
4.3. console.warn()	7
4.4. console.table()	8
4.5. Filtros y ejecución de JavaScript	8
5. Sources / Depurador	9
5.1. Breakpoints	9
5.2. Step over, Step into, Step out y Resume	9
5.3. Panel Scope	12
5.4. Panel Watch	13
5.5. Pause on exceptions	13
6. Network	13
6.1. Descarga de HTML, CSS y JavaScript	13
6.2. Tiempos de carga	14
6.3. Cabeceras y respuesta	14
7. Application	15
7.1. localStorage	15
7.2. sessionStorage	16
7.3. Cookies	16
8. Performance / Lighthouse	17
8.1. Grabación de una traza	17
8.2. Informe de Lighthouse	18
9. Conclusiones	19
10. Enlace a github	19

1. Introducción

En esta práctica he desarrollado una pequeña página web utilizando HTML, CSS y JavaScript con el objetivo de familiarizarme con las herramientas de desarrollo (DevTools) que ofrece el navegador. Para ello, he creado una página sencilla que contiene diferentes elementos HTML, estilos CSS y varias funciones JavaScript que permiten realizar distintas acciones.

El código está diseñado para poder probar y observar el funcionamiento de herramientas como Elements, Console y Sources, permitiendo modificar elementos, comprobar mensajes en la consola y depurar el código JavaScript mediante puntos de interrupción. También he incluido un error provocado intencionadamente para poder analizar cómo se muestra y cómo se puede localizar desde las DevTools.

Con esta práctica busco entender mejor cómo funcionan estas herramientas y aprender a utilizarlas para detectar y solucionar errores en una página web.

2. Banco de pruebas: código HTML, CSS y JavaScript

Cómo llegar: Abrir la carpeta del proyecto en el editor de código y abrir el archivo index.html. Localizar la parte donde se define la estructura HTML de la página.

Qué se ve: Se muestra el código HTML del banco de pruebas, con el título, el texto y los tres botones que después se utilizarán desde las DevTools.

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <title>Práctica DevTools</title>
    <link rel="stylesheet" href="style.css">
    <script src="script.js" defer</script>
  </head>
  <body>
    <h1>Práctica de DevTools</h1>
    <p>Esta página sirve para practicar las herramientas del navegador.</p>
    <button id="btnCalcular">Calcular</button>
    <button id="btnNumeros">Mostrar números</button>
    <button id="btnError">Provocar error</button>
    <p id="texto">Texto para practicar Elements</p>
  </body>
```

CSS

Cómo llegar: En el editor, abrir el archivo style.css enlazado desde index.html y situarse en las reglas que afectan a la página.

Qué se ve: Se observan las reglas CSS utilizadas para modificar el fondo, el título, los botones y el texto.

```
body {
  font-family: Arial;
  text-align: center;
  background-color: lightgray;
}

h1 {
  color: blue;
}

button {
  padding: 10px;
  margin: 5px;
}

#texto {
  color: green;
}
```

JS

Cómo llegar: Abrir script.js en el editor y localizar las funciones que controlan los botones.

Qué se ve: Se muestra el código JavaScript encargado de ejecutar las acciones de la página y de generar mensajes en la consola.

```
let numero = 5;

// Función 1
function calcular() {
  console.log("Estoy en calcular");

  let resultado = sumar(numero);

  console.log("El resultado es: " + resultado);
  console.warn("Estamos en el aviso")
}

// Función 2
function sumar(numero) {
  console.log("Estoy en sumar");

  let resultado = numero + 10;

  return resultado;
}

// Función 3
function mostrarNumeros() {
  for (let i = 0; i < 5; i++) {
    console.log("Número: " + i);
  }
}

// Error provocado
function provocarError() {
  throw new Error("Este es un error provocado");
}

//Tabla
console.table([1, 2, 3, 4, 5]);

// Botones
document.getElementById("btnCalcular").addEventListener("click", calcular);

document.getElementById("btnNumeros").addEventListener("click", mostrarNumeros);

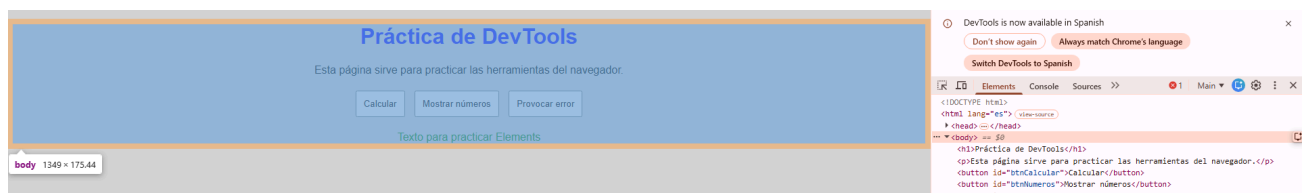
document.getElementById("btnError").addEventListener("click", provocarError);
```

3. Elements / Inspector

3.1. Inspección y edición del DOM

Cómo llegar: Abrir la página en Chrome, pulsar F12 o Ctrl+Shift+I y seleccionar la pestaña Elements. Si hace falta, usar el icono de selección de elementos y hacer clic sobre el elemento de la página.

Qué se ve: El panel Elements muestra el árbol HTML y el elemento <body> seleccionado. A la derecha se puede ver el HTML y las reglas asociadas.

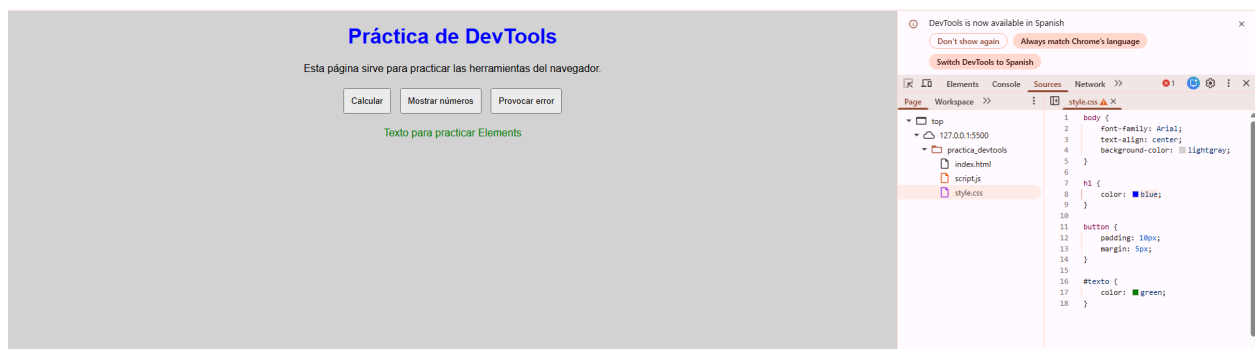


3.2. Edición del CSS en vivo

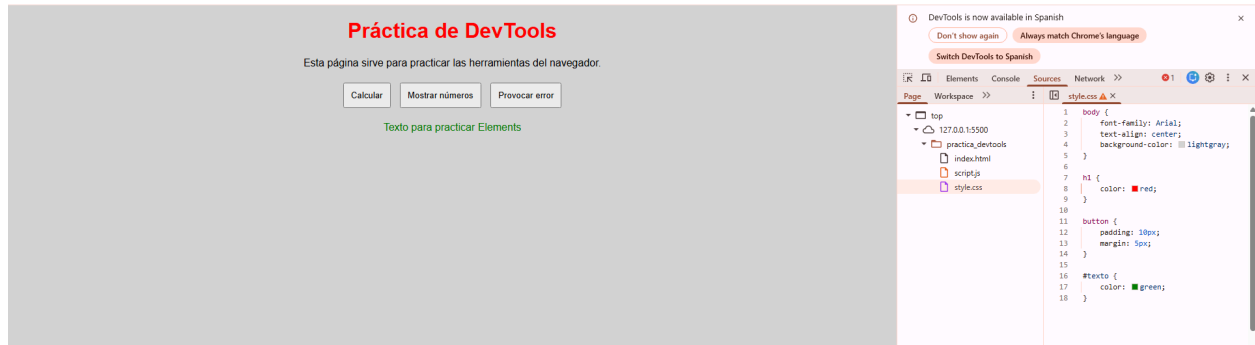
Cómo llegar: En Elements, seleccionar un elemento de la página y, en el panel Styles, localizar una propiedad CSS. Modificar su valor directamente.

Qué se ve: Esta captura representa el estado original de la página antes de aplicar el cambio de estilo.

Antes.



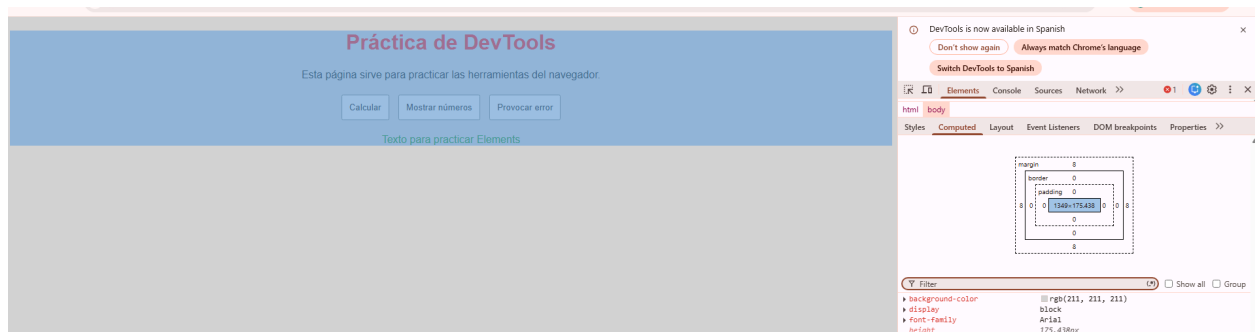
Después



3.3. Selector de elementos y estilos computados

Cómo llegar: En Elements, pulsar el icono de selección de elementos situado en la parte superior izquierda de DevTools y hacer clic sobre un elemento de la página. Después, revisar el panel de estilos y la información calculada.

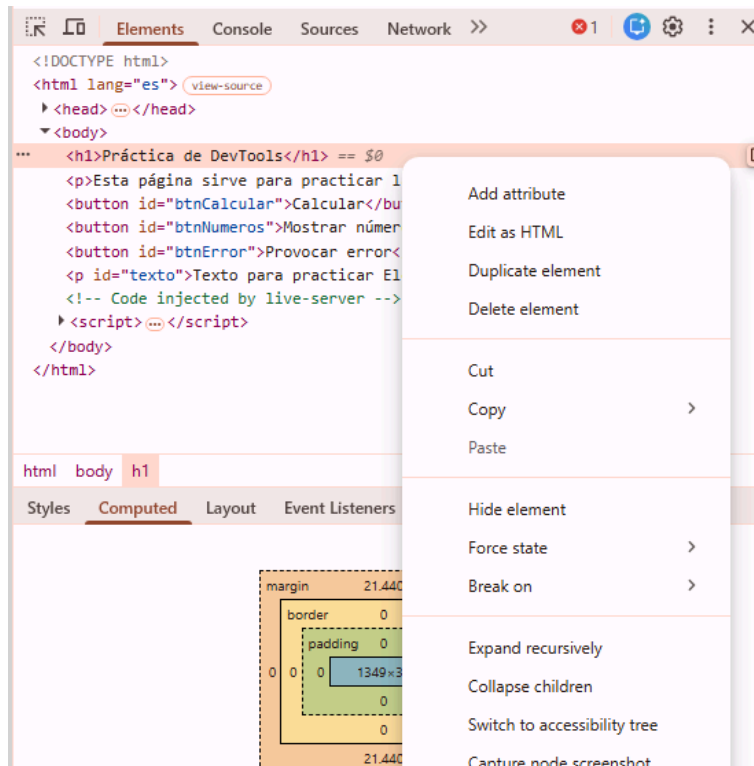
Qué se ve: El elemento seleccionado queda marcado en el DOM y DevTools permite consultar sus estilos. Los estilos computados representan los valores finales que el navegador está aplicando.



3.4. Edición de atributos y clases

Cómo llegar: En Elements, seleccionar el elemento y hacer clic con el botón derecho o editar directamente sus atributos y clases en el HTML mostrado.

Qué se ve: Se muestra el menú de opciones de edición del elemento y la posibilidad de modificar sus atributos o clases.



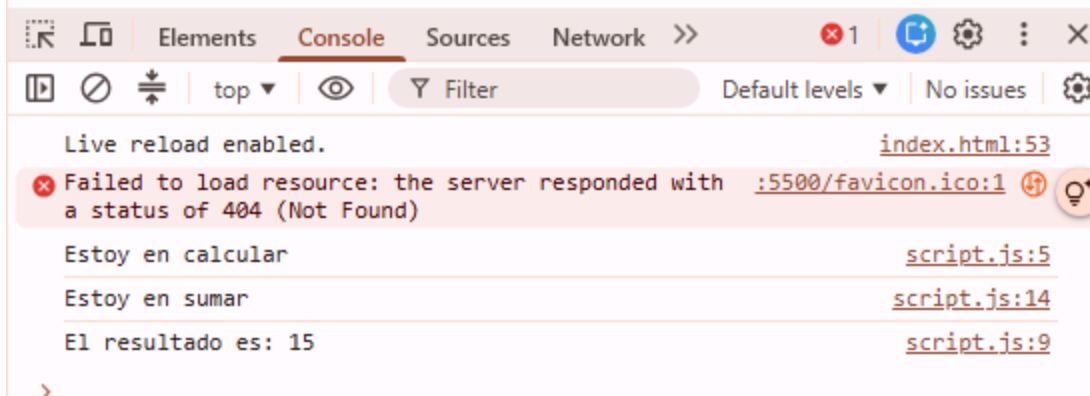
4. Console

Para abrirla, pulsar F12 o Ctrl+Shift+I y seleccionar Console. También se puede cambiar a Console desde el menú de pestañas de DevTools.

4.1. console.log()

Cómo llegar: Abrir Console y pulsar el botón de la página que ejecuta console.log().

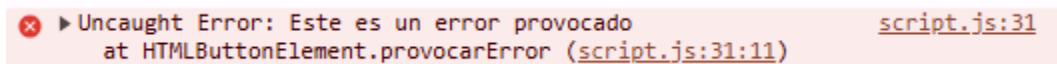
Qué se ve: La consola muestra los mensajes informativos generados por el script y permite relacionarlos con la línea del archivo JavaScript que los produjo.



4.2. console.error()

Cómo llegar: Con Console abierta, ejecutar la acción del proyecto que llama a console.error().

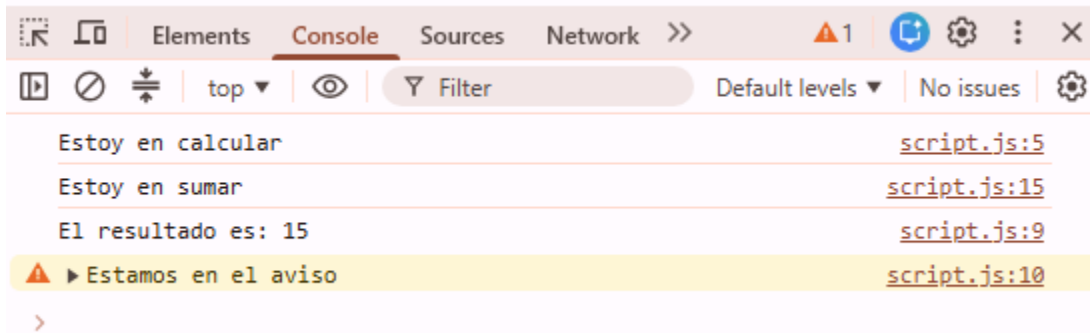
Qué se ve: Aparece un mensaje marcado como error en la consola.



4.3. console.warn()

Cómo llegar: Con Console abierta, ejecutar el botón o código que contiene console.warn().

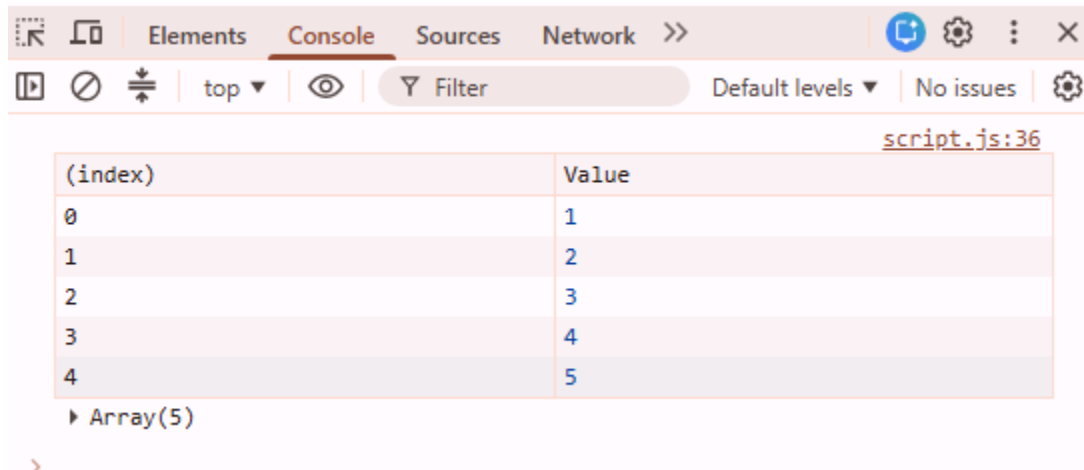
Qué se ve: Se muestra una advertencia diferenciada del resto de mensajes.



4.4. console.table()

Cómo llegar: Abrir Console y ejecutar la función del proyecto que utiliza console.table(), o escribir una llamada equivalente en la propia consola.

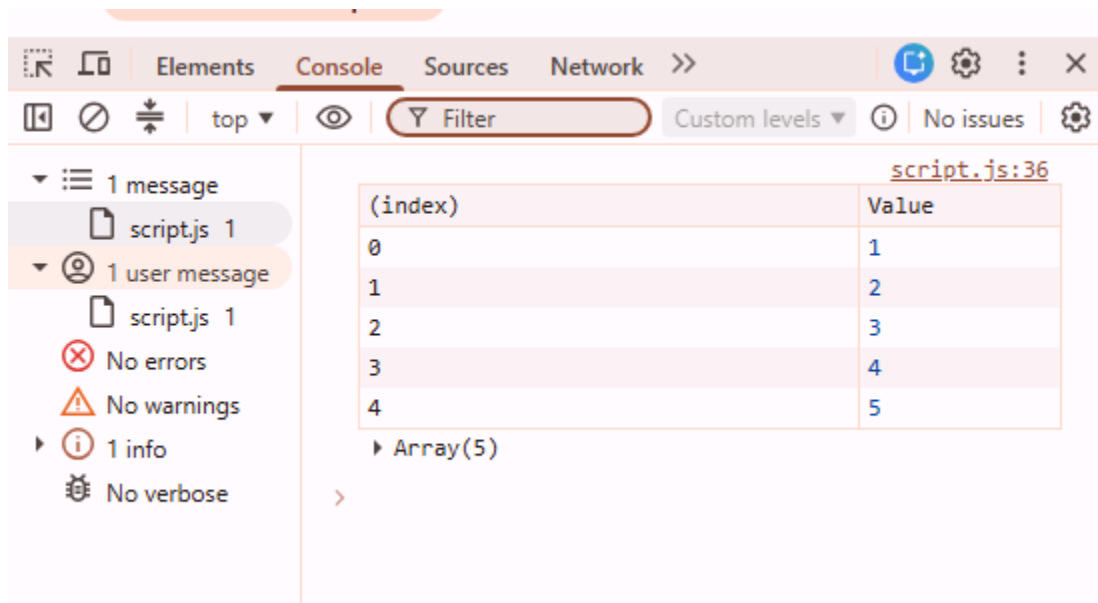
Qué se ve: Los datos aparecen organizados en filas y columnas, facilitando la lectura de los valores del array.



4.5. Filtros y ejecución de JavaScript

Cómo llegar: En Console, utilizar el cuadro Filter para mostrar determinados tipos de mensajes. Después, escribir una expresión JavaScript en la línea de entrada y pulsar Enter.

Qué se ve: La consola permite filtrar los mensajes y ejecutar JavaScript directamente sobre la página abierta.



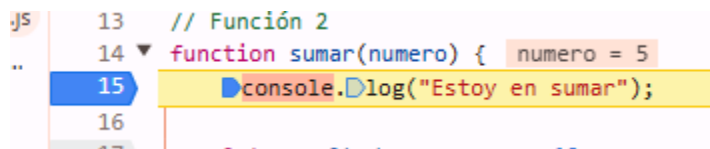
5. Sources / Depurador

Sources es el bloque central de la práctica. Para abrirlo, entrar en DevTools y seleccionar Sources. Desde aquí se puede localizar el archivo JavaScript, poner breakpoints y controlar la ejecución paso a paso.

5.1. Breakpoints

Cómo llegar: Abrir DevTools > Sources, localizar el archivo script.js en el árbol de archivos y hacer clic en el número de línea donde se quiere detener la ejecución.

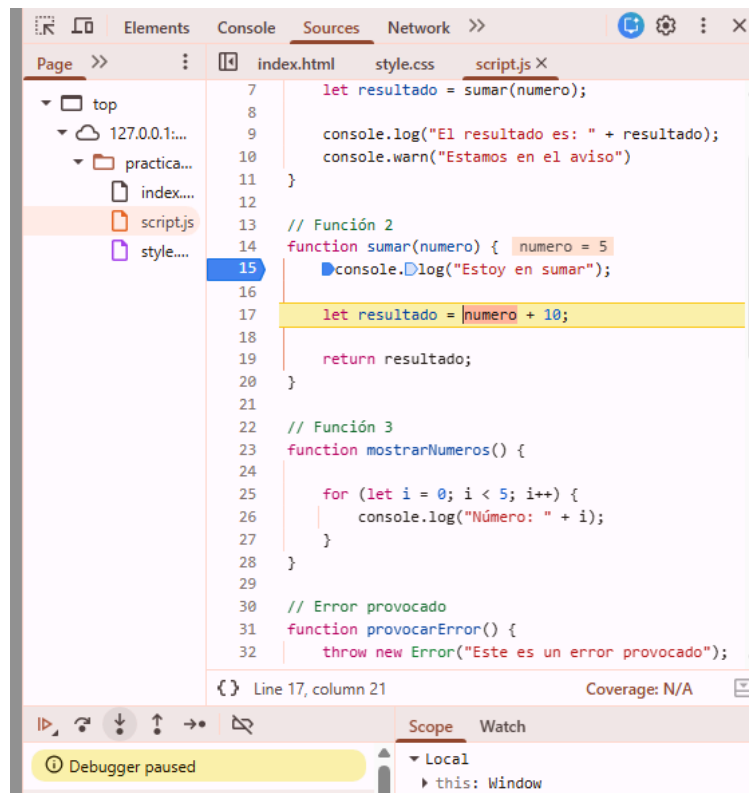
Qué se ve: El número de línea aparece marcado, indicando que se ha creado un breakpoint.



5.2. Step over, Step into, Step out y Resume

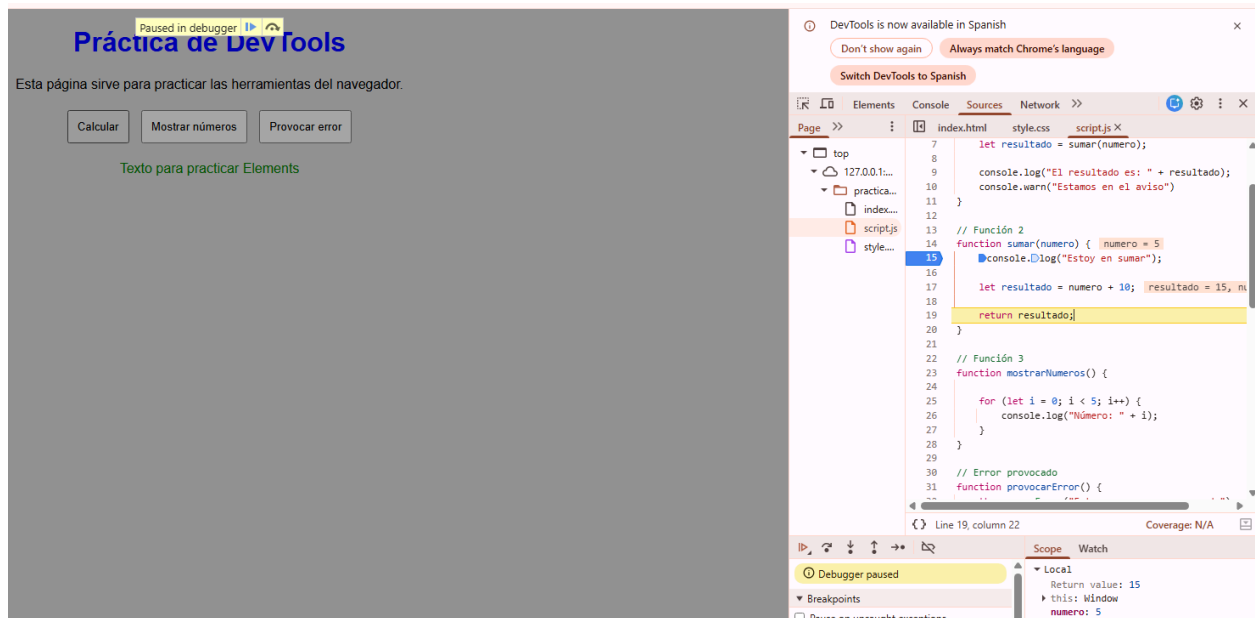
Cómo llegar: Con el breakpoint preparado, ejecutar desde la página la función calcular() para que la ejecución se detenga. Usar Step into cuando la línea llama a otra función.

Qué se ve: La ejecución está pausada dentro del flujo de llamadas y permite entrar en la función llamada.



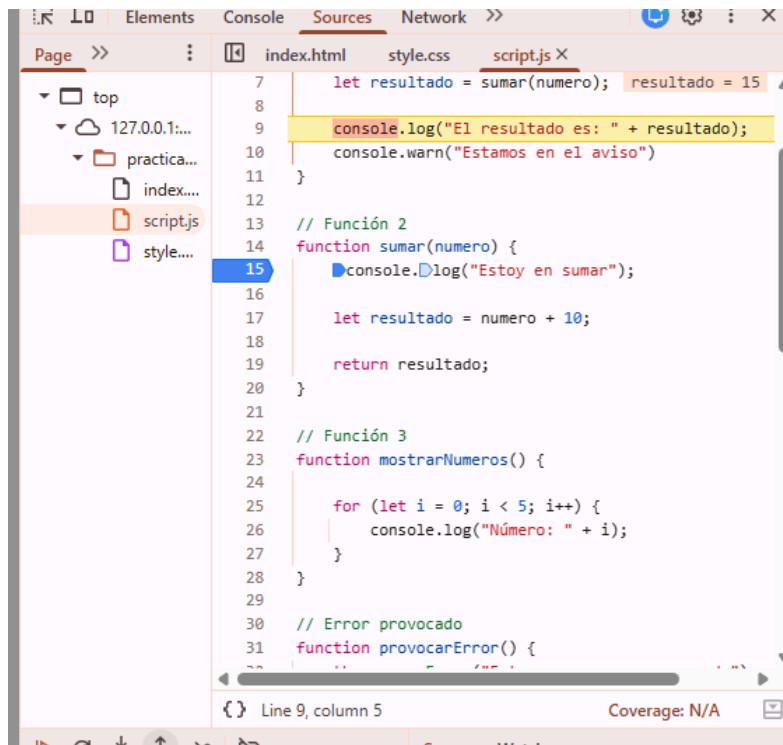
Cómo llegar: Continuar desde el punto de pausa y utilizar Step over para ejecutar la siguiente línea sin entrar en una función. Después, usar Step out cuando se quiera salir de la función actual.

Qué se ve: El depurador avanza línea a línea y permite controlar si se entra en una función o se continúa por encima de ella.



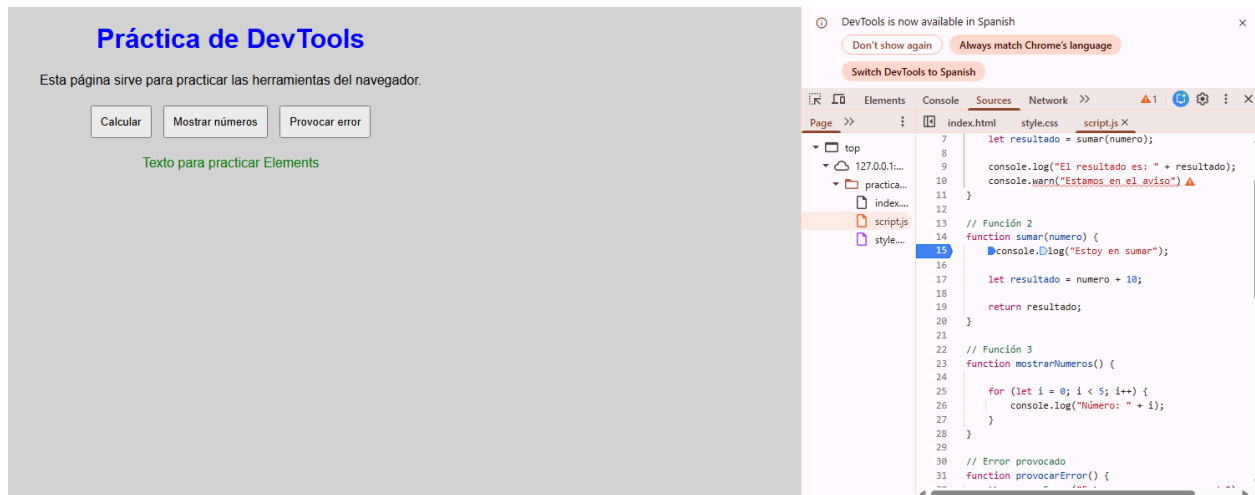
Cómo llegar: Después de detener la ejecución, observar el archivo script.js y utilizar los controles de depuración situados en la parte superior de Sources.

Qué se ve: Se muestra el código JavaScript con una línea activa y el punto actual de ejecución.



Cómo llegar: Pulsar Resume en los controles de depuración para continuar la ejecución hasta el siguiente breakpoint, una excepción o el final de la ejecución.

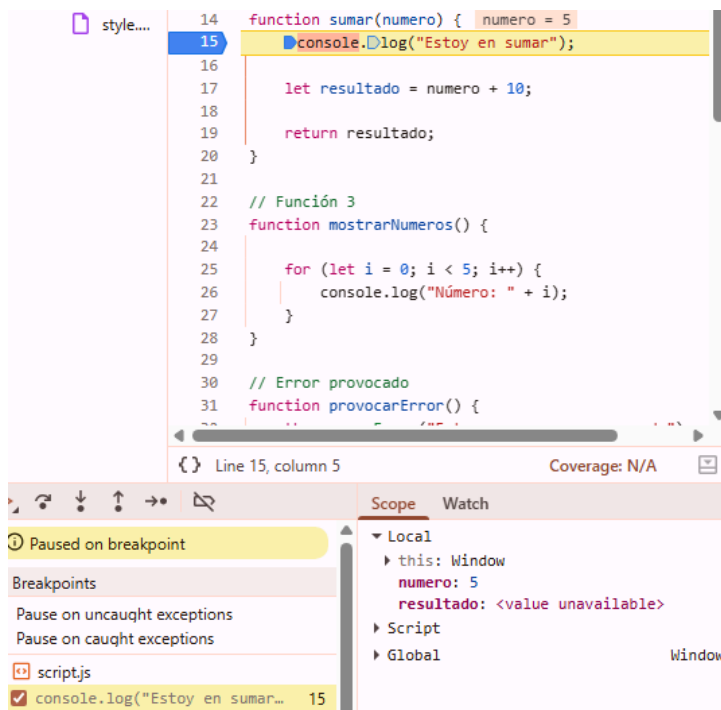
Qué se ve: La página vuelve a ejecutarse mientras Sources mantiene abierto el archivo JavaScript.



5.3. Panel Scope

Cómo llegar: Detener la ejecución en un breakpoint y abrir el apartado Scope en el panel derecho de Sources.

Qué se ve: Scope muestra las variables disponibles en el punto exacto en el que se ha detenido el programa, incluyendo los ámbitos que correspondan a la ejecución.



5.4. Panel Watch

Cómo llegar: Con la ejecución pausada en Sources, abrir Watch y añadir una variable o expresión relacionada con el código, por ejemplo el resultado que se está calculando.

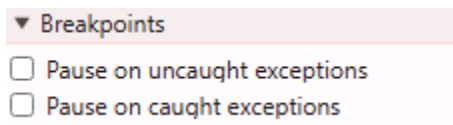
Qué se ve: Watch muestra la expresión añadida y su valor mientras se depura.



5.5. Pause on exceptions

Cómo llegar: En Sources, activar la opción Pause on exceptions y ejecutar desde la página la función provocarError().

Qué se ve: La opción de pausa ante excepciones queda activada para que DevTools detenga automáticamente la ejecución cuando se produzca el error.



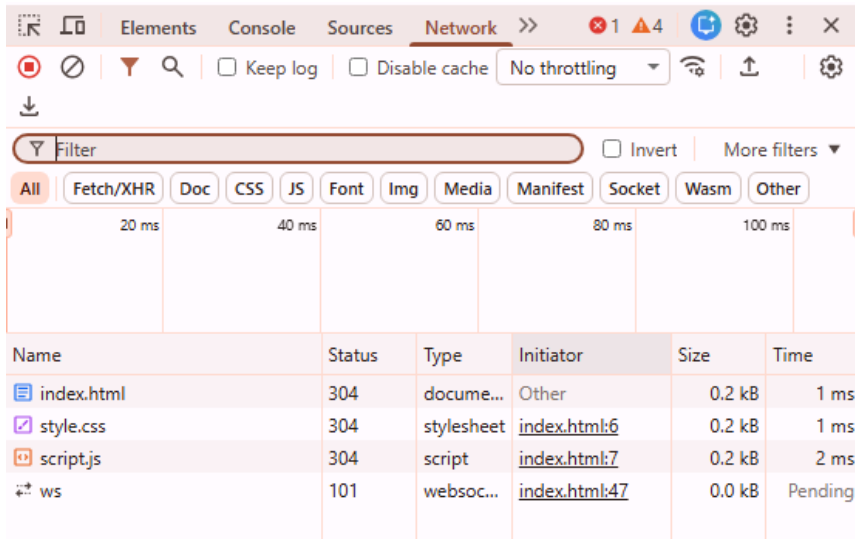
6. Network

Para abrir Network, entrar en DevTools y seleccionar la pestaña Network. Conviene recargar la página con Network abierto para registrar desde el principio las peticiones y recursos.

6.1. Descarga de HTML, CSS y JavaScript

Cómo llegar: Abrir DevTools > Network y recargar la página. En la lista de peticiones, localizar los recursos HTML, CSS y JavaScript del proyecto.

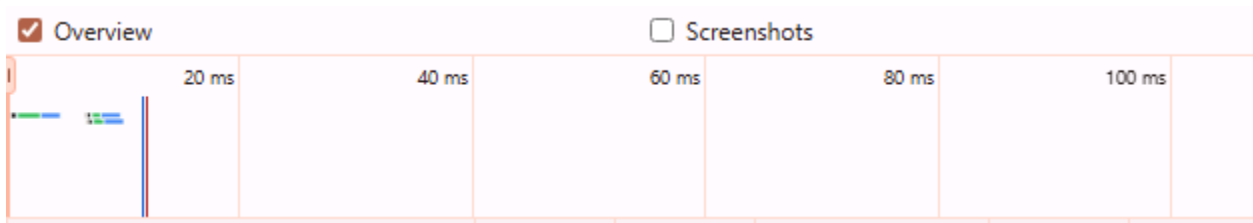
Qué se ve: La tabla de Network muestra los recursos solicitados por el navegador y datos básicos de cada petición.



6.2. Tiempos de carga

Cómo llegar: Con Network abierto, recargar la página y observar las columnas relacionadas con tiempo y tamaño de los recursos.

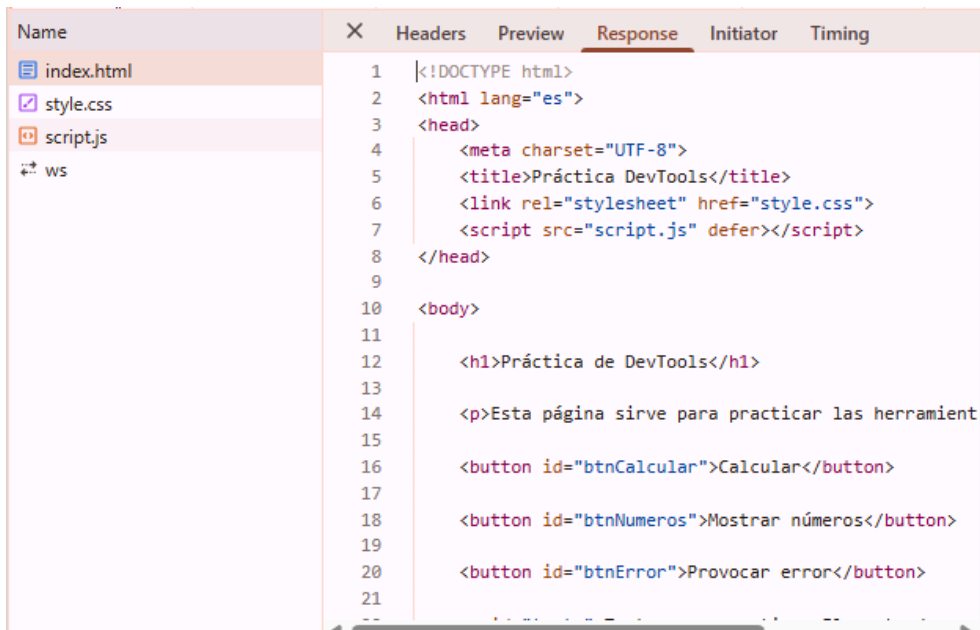
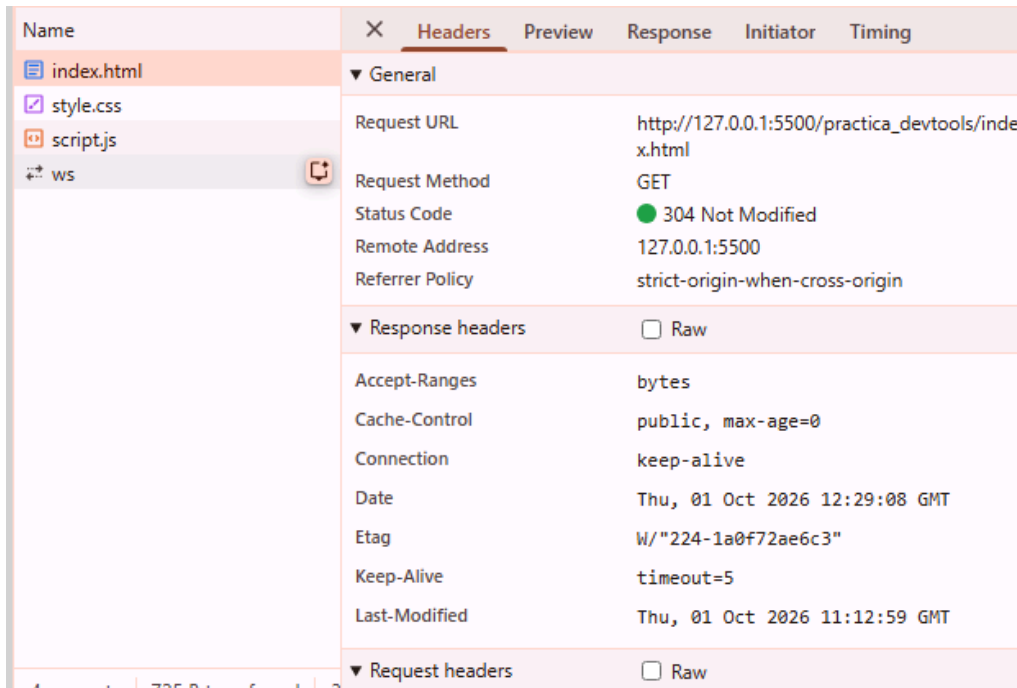
Qué se ve: La captura muestra la información temporal de la carga de los recursos.



6.3. Cabeceras y respuesta

Cómo llegar: En Network, seleccionar una petición concreta y abrir las pestañas de detalles, especialmente Headers y Response.

Qué se ve: Headers contiene información asociada a la petición y a la respuesta; Response muestra los datos devueltos por el servidor.



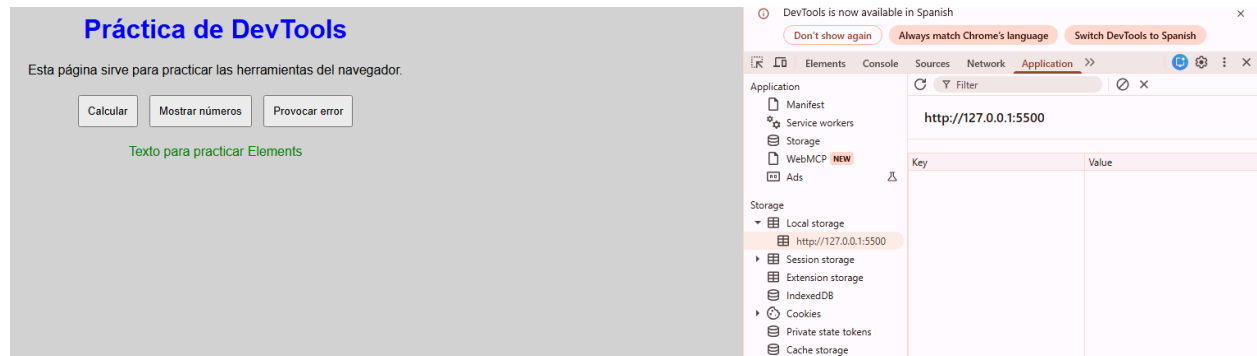
7. Application

Para abrir Application, seleccionar esta pestaña en DevTools. Si no aparece directamente, utilizar el menú de pestañas adicionales (>>). Desde aquí se consultan los datos almacenados por la página.

7.1. localStorage

Cómo llegar: Abrir DevTools > Application y, en el panel izquierdo, desplegar Local Storage. Seleccionar el origen de la página.

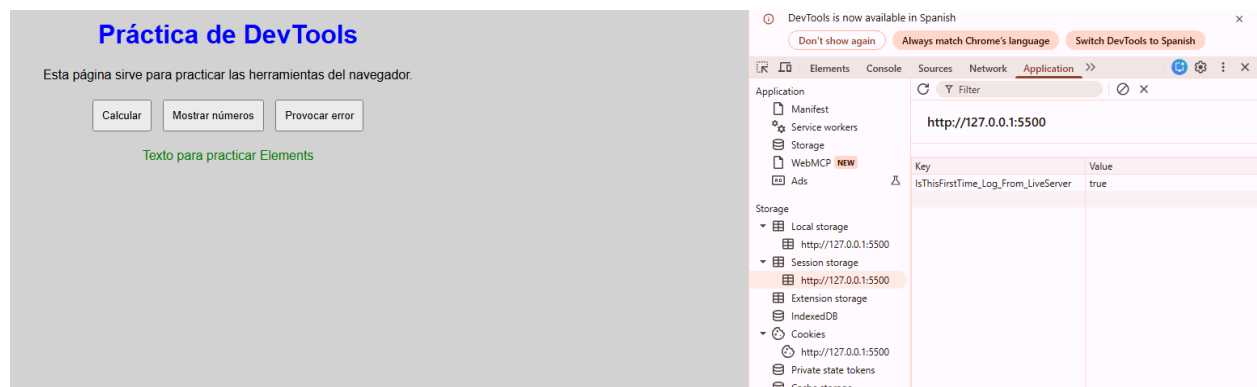
Qué se ve: Se muestra el apartado de almacenamiento local asociado al sitio.



7.2. sessionStorage

Cómo llegar: En Application, desplegar Session Storage y seleccionar el origen de la página.

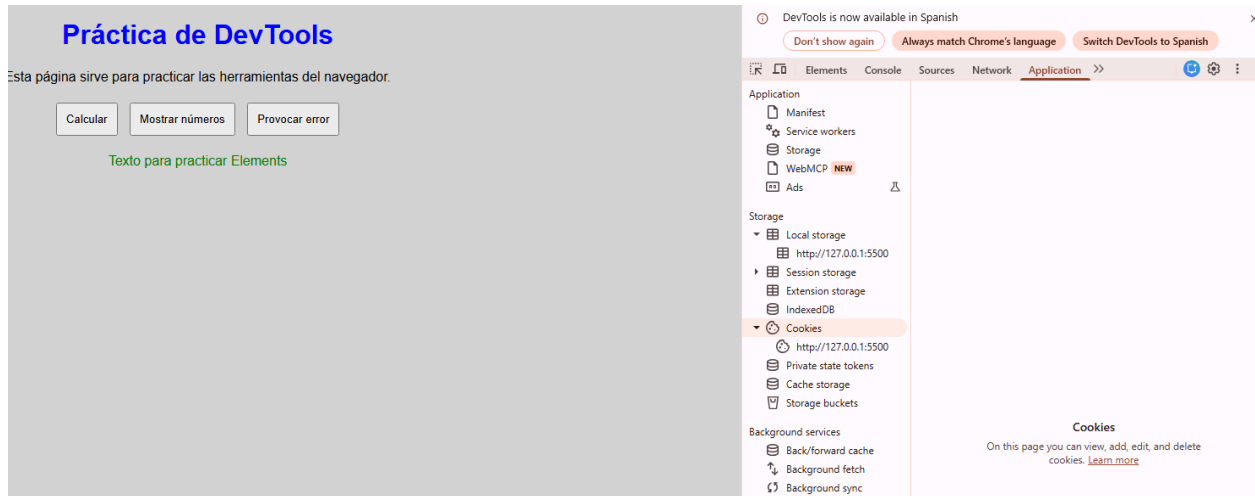
Qué se ve: Se muestra el almacenamiento asociado a la sesión actual del navegador.



7.3. Cookies

Cómo llegar: En Application, desplegar Cookies y seleccionar el dominio correspondiente a la página.

Qué se ve: Se muestran las cookies almacenadas para el sitio y sus diferentes propiedades.



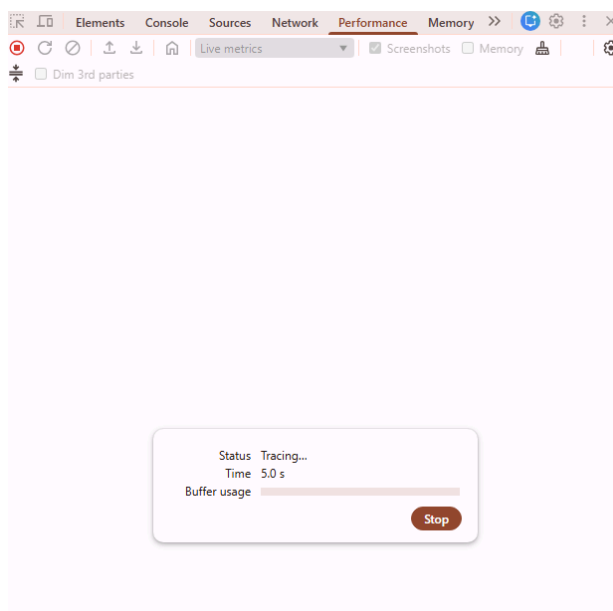
8. Performance / Lighthouse

Estas herramientas permiten analizar el rendimiento de la página. Performance sirve para grabar una traza de ejecución, mientras que Lighthouse genera un informe con diferentes categorías de evaluación.

8.1. Grabación de una traza

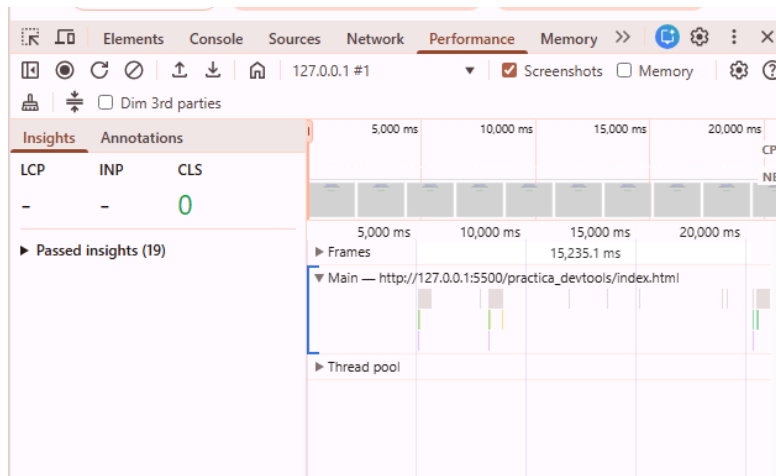
Cómo llegar: Abrir DevTools > Performance y pulsar el botón de grabación. Realizar una acción en la página, como cargarla o pulsar un botón, y detener la grabación.

Qué se ve: Se observa el panel de Performance preparado para registrar y analizar la actividad de la página.



Cómo llegar: Después de detener la grabación, revisar la línea temporal y los eventos registrados por Performance.

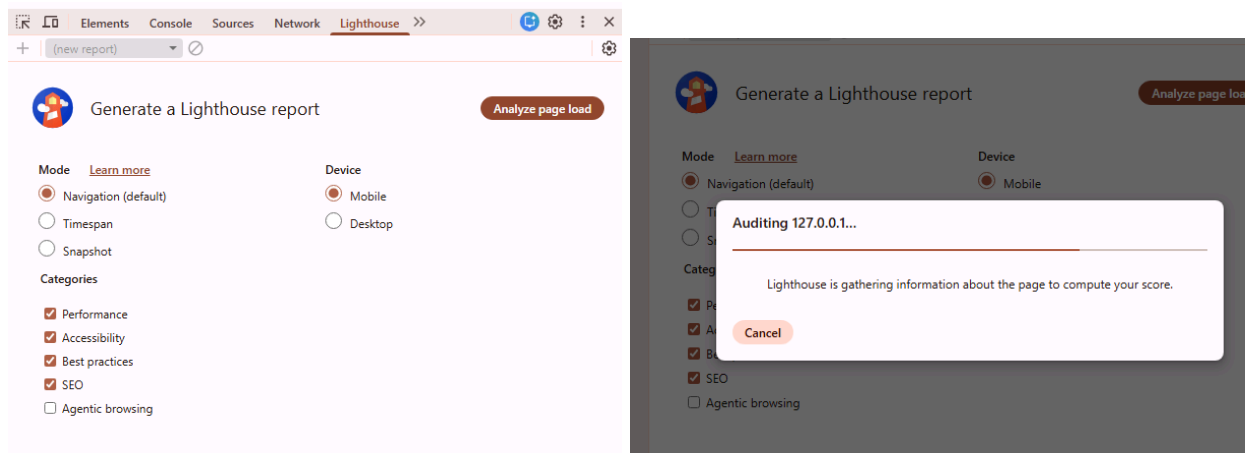
Qué se ve: La traza muestra la actividad registrada durante el intervalo analizado.



8.2. Informe de Lighthouse

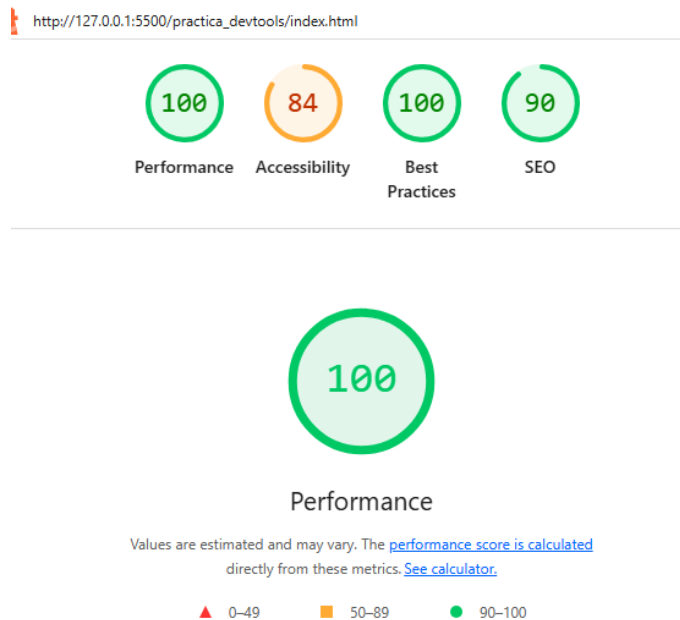
Cómo llegar: Abrir Lighthouse desde DevTools, seleccionar las categorías que se quieran analizar y generar el informe sobre la página actual.

Qué se ve: Lighthouse muestra las opciones de auditoría y permite iniciar el análisis.



Cómo llegar: En el informe generado, seleccionar una categoría o revisar las recomendaciones que aparecen debajo de las puntuaciones.

Qué se ve: La captura muestra una de las puntuaciones obtenidas en el informe.



9. Conclusiones

En esta práctica he aprendido a utilizar las principales herramientas de desarrollo del navegador y a entender para qué sirve cada una. También he podido comprobar cómo las DevTools ayudan a encontrar errores y entender mejor el funcionamiento de una página web.

La parte de Sources me ha servido para aprender a utilizar breakpoints y ejecutar JavaScript paso a paso, mientras que con Console, Elements, Network y Application he podido analizar diferentes partes de la página.

En general, esta práctica me ha ayudado a familiarizarme con las DevTools y a comprender que son una herramienta muy útil para desarrollar y solucionar problemas en aplicaciones web.

10. Enlace a github

<https://github.com/crodhid>